Dependency-Aware Scheduling for Efficient Knowledge Graph Updates

Shivam Arora, Dimash Umirbayev, Aman Soni, Leonard Zhang

UNIVERSITY OF MICHIGAN

ABSTRACT

This project introduces a **dependency-aware scheduling system** to accelerate LLM-driven knowledge graph updates.

By modeling semantic dependencies as a **Directed Acyclic Graph (DAG)**, the system enables safe parallelism and efficient batching. This approach maximizes GPU utilization and context reuse while ensuring the semantic consistency of an agent's long-term memory

MOTIVATION

- Modern AI agents increasingly rely on knowledge graphs to maintain long-term memory and structured world knowledge.
- To remain useful, these systems must process high volumes of incoming information streams with minimal delay.
- However, existing LLM serving systems typically assume independent requests and do not account for the semantic dependencies required for memory evolution, creating a tension between processing speed and data integrity.

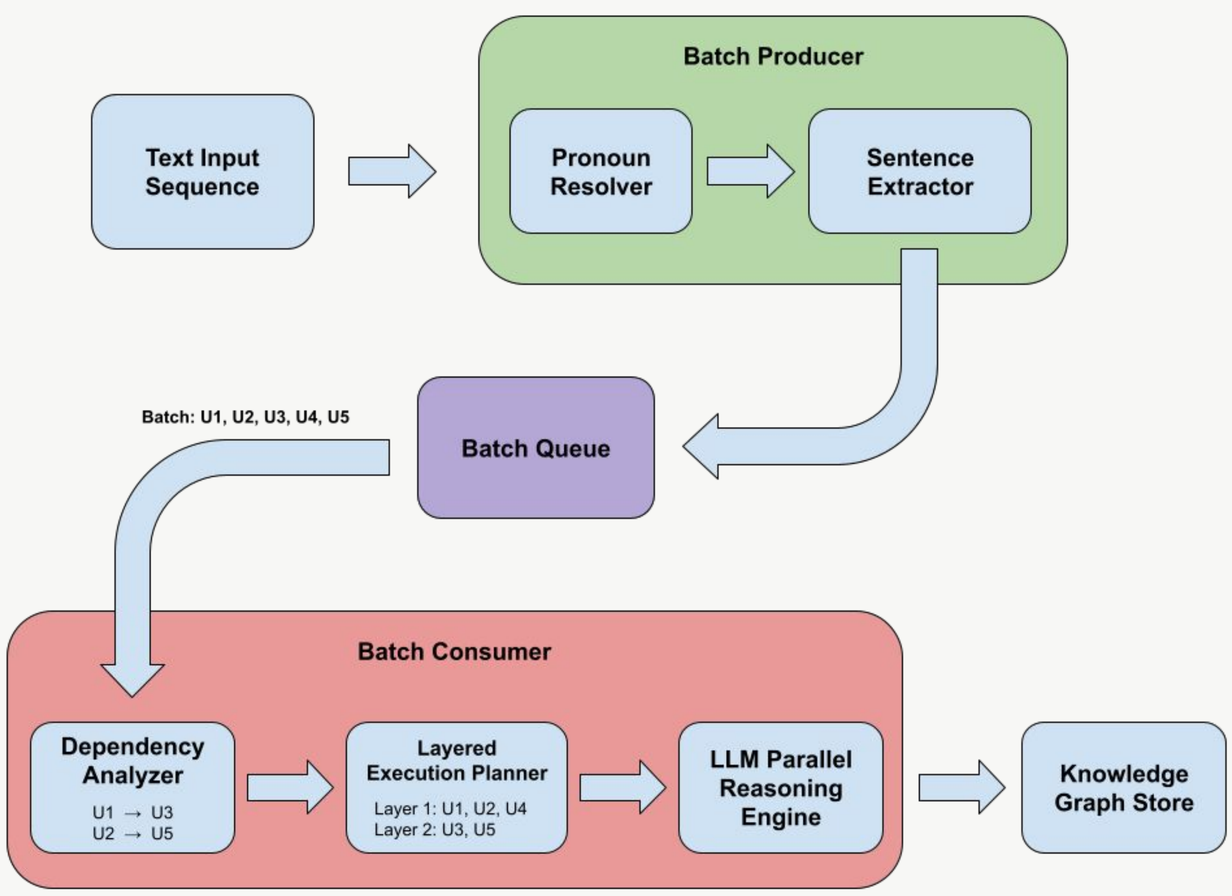
PROBLEM STATEMENT

- Traditional knowledge graph updates rely on **sequential execution** to ensure consistency, which results in low GPU utilization, limited throughput, and high latency.
- **Naive batching** increases performance but risks "staleness" and inconsistent memory states due to dependency-agnostic execution.
- The key challenge is: **How can we execute KG updates in parallel while preserving memory correctness?**

SYSTEM COMPONENTS

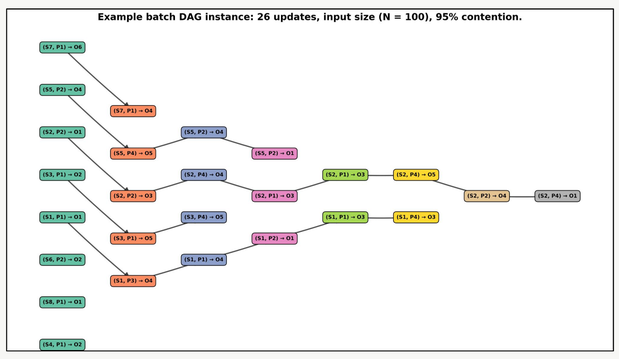
- **Pronoun Resolver**: replaces pronouns like "she" or "it" into the proper noun using a small language model
- **Sentence Extractor**: convert a sentence into a KG update by extracting the subject, predicate, and object of a sentence
  - ex: John lives in London → (John, lives) = London
- **Dependency Analyzer**: determines whether 2 updates conflict by comparing the subject of each update
- **Layered Execution Planner**: layers updates based on the result of the dependency analyzer. Conflicting updates must be placed in different layers while non-conflicting updates can be in the same layer
- **LLM Parallel Reasoning Engine**: decides whether to apply each update by inspecting each (subject, predicate) value. It also reasons over changes using the current snapshot and incoming updates
- **Knowledge Graph Store**: a knowledge graph implemented either in Neo4j or in Memory

SYSTEM ARCHITECTURE



EXECUTION WORKFLOW

Producer/Consumer pipeline: Updates enter through a producer and are drained by a consumer.
Batching: Requests are grouped to amortize overhead and raise throughput.
Dependency-Aware scheduling: Conflicting updates are separated; independent ones run together.
Correctness guard: Final execution preserves sequential semantics or consistency.



EXPERIMENT SETUP

We executed a grid of **64 independent workloads**:

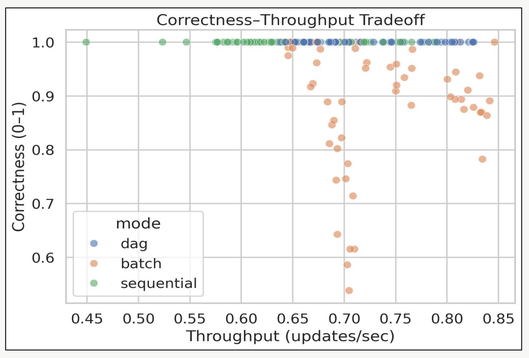
WORKLOAD DIVERSITY
Natural Language (Real) vs. Abstract ID streams.

CONTENTION
Scaling "**Hot Ratios**" from 0.1 to 0.95 to simulate extreme data contention.

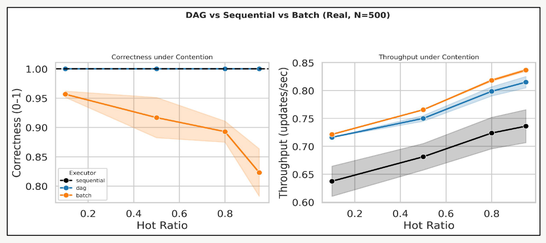
THE ENGINE
Qwen-2.5-7B hosted on an **NVIDIA A40** via vLLM for native batch inference.

KEY RESULTS

- **DAG** delivers the **best speed correctness tradeoff**.



- **DAG preserves correctness with higher throughput** than sequential.
- **Batch boosts throughput but breaks correctness** under contention.



FUTURE STEPS

- **Semantic Dependency Detection:** Map synonymous actions to single state locks, preventing race conditions in natural language.
- **Adaptive Batch Windowing**: Implement Dynamic Throttling to optimize GPU throughput.
- **Scaling extraction to Video Streams**: A DAG that can sequence temporal visual events.